Enhanced Mapping Using Infrastructure-Based Camera Networks

Hao Nguyen Thien, Dung Bui Manh, and Luc Xuan Le

***Abstract*—SLAM systems accumulate drift over long trajectories, degrading trajectory accuracy and map consistency—particularly for 2D LiDAR systems in featureless indoor environments. Infrastructure-assisted approaches typically register fixed cameras onto a pre-built SLAM map, so any drift in that map biases subsequent camera poses. This paper presents a method that jointly estimates the environment map and the poses of ceiling-mounted cameras, enabling cameras to act as global anchors without a separate registration stage. When a camera observes the robot, a visual constraint is inserted into the pose graph, tying the robot trajectory to the camera's map-frame location. Unlike loop closure alone, these repeated absolute constraints suppress drift in visually ambiguous and revisited areas. The approach integrates into a graph-based SLAM back-end via pose-graph optimisation with no modifications to the core pipeline. Experiments in simulated indoor environments demonstrate improved map consistency and reduced absolute trajectory error compared with a LiDAR-only baseline, at modest additional computational cost.**

***Index Terms*—SLAM, visual constraints, pose-graph optimisation, fixed cameras**

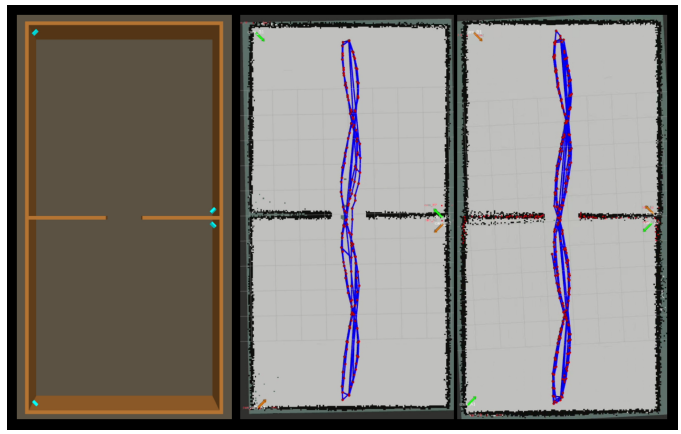


Fig. 1: Qualitative overview of the 2-room scenario. **Left:** Gazebo environment. **Middle:** LiDAR-only SLAM — a camera marker is misregistered into the neighbouring room. **Right:** Proposed method — camera nodes remain in the correct room corners.

I. Introduction

Autonomous mobile robots operating in indoor environments must maintain an accurate global pose estimate to navigate safely, revisit places reliably, and build maps that remain useful over long deployments. At a high level, SLAM can be interpreted as a probabilistic estimation problem in which the robot seeks to infer its trajectory and the map from a stream of noisy controls and sensor measurements. Classical Bayesian filtering formulations, including Kalman-filter variants and particle-filter methods such as FastSLAM, update this belief recursively as new observations arrive [1], [2]. This recursive structure makes them attractive for online operation and for tightly coupled multi-sensor fusion, where camera, LiDAR, and IMU measurements must be integrated at different rates within a common state-estimation framework [3]. At the same time, filtering approaches commit to local updates as data arrive, which makes it less natural to incorporate long-range dependencies, delayed loop closures, and global consistency corrections than in full smoothing or factor-graph formulations.

Graph-based SLAM addresses these limitations by representing robot poses and map variables as nodes in a sparse factor graph and encoding odometry, scan-matching, and loop-closure measurements as constraints between them. Inference then becomes a nonlinear optimisation problem over the full graph, which enables robust global refinement and has made graph-based methods a dominant formulation for modern SLAM back-ends [4], [5]. This is also the setting adopted by the 2D LiDAR SLAM backend used in this work [6]. However, for 2D LiDAR-based systems operating in large, repetitive, or featureless environments such as corridors, open halls, or sparsely structured rooms, drift can remain significant even after loop closures because local scan geometry is often insufficiently distinctive and the available constraints are too weak to anchor the map globally.

One way to supply stronger global information is to exploit an infrastructure camera network. Fixed ceiling- or wall-mounted cameras observe the robot from stable viewpoints that are decoupled from the robot's own motion and can therefore provide complementary constraints to onboard LiDAR. Recent work such as YOWO [7] has shown that jointly mapping an indoor scene and registering ceiling-mounted cameras can improve both scene reconstruction and camera registration. That observation is directly relevant here: when cameras are registered only after a SLAM map has already been created, any residual map drift is inherited by the camera poses and later reused as a biased reference. At the same time, YOWO focuses on RGB-D scene capture and scene-layout registration, whereas our target problem is 2D LiDAR SLAM supported by sparse visual anchor constraints from fixed infrastructure

The authors are with the Faculty of Electronics and Telecommunications, University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam. E-mail: {thienhaonguyen07, buimanhdung, email}@example.com

cameras during mapping.

This paper explores that gap by augmenting a 2D LiDAR pose graph with camera-derived constraints so that infrastructure observations contribute to global consistency during optimisation rather than only after a separate manual registration stage.

Figure 1 illustrates this contrast: without camera constraints the pose-graph drifts and misregisters a camera marker into the wrong room, whereas the proposed augmentation keeps all camera nodes consistent with their correct room corners after optimisation.

We address this gap by contributing:

- A constraint formulation that injects a camera-to-robot relative observation into a Ceres [8] pose graph whenever the robot is visually detected, with per-axis covariance weighting.
- A camera-based robot pose estimation pipeline that injects visual anchor constraints into the pose graph of the LiDAR SLAM backend without replacing the underlying 2D LiDAR SLAM pipeline.
- An experimental evaluation in simulated 2-room, 5-room, and 10-room ring environments demonstrating measurable reduction in absolute trajectory error compared to a LiDAR-only baseline.

The remainder of this paper is organised as follows. Section II reviews related work. Section III describes the constraint formulation and system architecture. Section IV details the experimental setup. Section V presents and discusses results. Section VI concludes.

II. Related Work

A. Graph-Based SLAM

Graph-based SLAM [4] represents the robot trajectory as a factor graph whose nodes are poses and whose edges encode odometry or loop-closure constraints. Optimisation with Gauss–Newton or Levenberg–Marquardt [8] minimises the total squared residual. A representative implementation [6] extends this with lifelong and multi-session mapping, operating in synchronous or asynchronous scan-matching modes. A key limitation of purely LiDAR-based graphs is that drift accumulates in featureless corridors where loop-closure opportunities are rare.

B. Infrastructure Camera Localisation

Infrastructure-mounted cameras offer a complementary sensing modality for indoor localization because they observe the robot from viewpoints that are independent of the robot's onboard motion. However, as emphasized in YOWO [7], reliable registration of ceiling-mounted cameras to the scene layout remains the key bottleneck. The related literature highlighted by YOWO falls into three main groups.

The first group uses SLAM or visual localization to reconstruct an indoor scene and then localize a query camera against that scene model. Representative methods such as InLoc [9] and hierarchical visual localization [10] achieve strong performance in indoor visual localization, but they rely heavily on static scene features and can degrade under viewpoint disparity, repeated structures, and ambiguous textures. These challenges become more severe when matching ceiling-mounted cameras against egocentric observations collected from very different perspectives.

The second group estimates relative poses between infrastructure cameras by tracking mobile keypoints, often from a human moving through the scene. Examples include extrinsic camera calibration from a moving person [11] and online marker-free extrinsic calibration using person keypoints [12]. These methods are robust to cross-view ambiguity because they rely on dynamic agents rather than static scene appearance, but they typically recover only relative camera geometry and do not automatically align the resulting camera poses to a globally consistent scene map.

The third group studies collaboration between egocentric and external cameras. Works such as EgoHumans [13] show that combining egocentric and externally mounted views improves downstream perception tasks, but camera registration is generally handled beforehand as a separate step. YOWO is important because it explicitly connects these three lines of research and argues for a unified formulation that jointly maps the scene and registers ceiling-mounted cameras.

Our work shares YOWO's motivation that post-hoc registration is fragile, but it targets a different estimation setting: instead of RGB-D scene reconstruction with collaborative ego and overhead video, we focus on 2D LiDAR pose-graph SLAM and use fixed cameras as static anchor sources for reducing drift in the robot trajectory.

C. Constraint Injection in SLAM

Suppressing cumulative drift by injecting external constraints into the SLAM optimization graph is a well-established idea. Two common sources of such constraints are semantic landmarks and infrastructure-based ranging anchors. For example, fusing UWB range measurements with short-range 2D LiDAR can add global distance constraints that reduce the position drift and map divergence of standalone 2D SLAM [14]. However, UWB-based approaches remain sensitive to indoor non-line-of-sight and multipath effects, and their performance often depends on a sufficiently dense, pre-installed anchor infrastructure [15], [16].

Semantic or vision-aided SLAM methods inject higher-level environmental cues to improve loop closure and map consistency. For instance, low-cost LiDAR-vision fusion has been used to build 2.5D maps and stabilize localization by combining complementary geometric and visual information [17]. Yet these methods can become computationally demanding and may degrade in environments with repetitive appearance, dynamic objects, or weakly distinctive textures.

Structurally, our method is closest to SLAM systems that bound drift by adding absolute reference measurements to a pose graph. In outdoor mapping, Global Positioning System (GPS) observations play this role by constraining the long-term drift of LiDAR odometry. Surveys of GPS-denied LiDAR

SLAM make clear, however, that such global fixes are unavailable in indoor facilities and deep urban canyons, where signal attenuation or occlusion renders satellite positioning unreliable or unusable [16]. Our method follows the same design principle of augmenting the graph with external absolute references, but replaces GPS or UWB range anchors with camera-derived robot observations from fixed infrastructure cameras.

III. Methodology

A. Notation

We formulate the problem on a pose graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where vertices represent poses and edges represent spatial constraints. The vertex set is partitioned into robot poses $\mathcal{V}_{\mathrm{r}} = \{\mathbf{x}_t\}$ and anchor poses $\mathcal{V}_{\mathrm{a}} = \{\mathbf{x}_{\mathcal{C}_i}\}$. Both robot poses and camera poses are optimisation variables in the augmented graph. Each robot state $\mathbf{x}_t \in SE(2)$ denotes the robot pose at time index t, and each $\mathbf{x}_{\mathcal{C}_i} \in SE(2)$ denotes the map-frame pose of infrastructure camera $\mathcal{C}_i$. The edge set is partitioned into pose-pose constraints $\mathcal{E}_{\mathrm{pp}}$ and pose-anchor constraints $\mathcal{E}_{\mathrm{pa}}$. In our case, $\mathcal{E}_{\mathrm{pp}} = \mathcal{E}_{\mathrm{odom}} \cup \mathcal{E}_{\mathrm{loop}}$ contains odometry and loop-closure edges, and $\mathcal{E}_{\mathrm{pa}} = \mathcal{E}_{\mathrm{cam}}$ contains camera-derived anchor edges. Each edge carries a relative measurement $\mathbf{z}_{ij}$ or $\mathbf{z}_{ik}$ and an associated information matrix. For a residual vector $\mathbf{r}$ weighted by information matrix $\mathbf{\Omega}$, we use the notation $\|\mathbf{r}\|^2_{\mathbf{\Omega}} = \mathbf{r}^\top \mathbf{\Omega} \mathbf{r}$.

B. Pose-Graph Preliminaries

Pose-graph SLAM can be viewed as a weighted directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \omega)$, in which each pose node belongs to $\mathcal{V}$ and each edge in $\mathcal{E}$ encodes a relative measurement between two poses [18]. For the robot trajectory, let $\mathcal{V}_{\mathrm{r}} = \{\mathbf{x}_1, \ldots, \mathbf{x}_{n_p}\}$ denote the set of n_p robot poses and let $\mathcal{E}_{\mathrm{pp}} = \{e_k\}_{k=1}^m$ denote the set of m pose-pose constraints. Each edge $e_k = (i, j)$ carries a relative measurement $\mathbf{z}_{ij}$ and a positive weight $\omega_k > 0$, whose magnitude is inversely related to the measurement variance and therefore reflects the reliability of that constraint.

The graph structure can be encoded by the incidence matrix $A \in \{-1, 0, 1\}^{n_p \times m}$. For edge $e_k = (i, j)$, the k-th column of A has entry -1 in row i, +1 in row j, and zeros elsewhere. If the edge weights are collected in the diagonal matrix $W = \mathrm{diag}(\omega_1, \ldots, \omega_m)$, then the weighted graph Laplacian is

$$L_w = AWA^\top. \tag{1}$$

This matrix captures both graph topology and measurement confidence, and it is useful for analyzing how strongly the pose graph is constrained.

In the multiple-anchor setting, a subset of nodes is designated as anchors. In our formulation, the infrastructure cameras form the anchor-node set $\mathcal{V}_{\mathrm{a}} = \{\mathbf{x}_{\mathcal{C}_i}\}$. Removing the rows of A that correspond to anchored nodes yields a reduced incidence matrix $A_{\setminus \mathcal{V}_{\mathrm{a}}}$, and the corresponding reduced weighted Laplacian becomes

$$L_{w,\setminus \mathcal{V}_{\mathrm{a}}} = A_{\setminus \mathcal{V}_{\mathrm{a}}} W A^\top_{\setminus \mathcal{V}_{\mathrm{a}}}. \tag{2}$$

This viewpoint clarifies why anchor placement matters: informative anchors alter the reduced graph structure and improve the observability of the remaining unanchored robot poses. In our case, camera detections provide pose-anchor measurements $\mathbf{z}_{ik}$ between robot pose $\mathbf{x}_i$ and anchor pose $\mathbf{x}_{\mathcal{C}_k}$, which are incorporated alongside the standard pose-pose constraints.

Let $\mathbf{X} = \{\mathbf{x}_t\}$ denote the robot trajectory and $\mathbf{C} = \{\mathbf{x}_{\mathcal{C}_i}\}$ the set of camera poses to be optimized jointly. The backend solves the nonlinear least-squares problem

$$\begin{aligned}(\mathbf{X}^\star, \mathbf{C}^\star) = \arg\min_{\mathbf{X},\mathbf{C}} & \sum_{(i,j)\in\mathcal{E}_{\mathrm{pp}}} \|\mathbf{r}_{ij}(\mathbf{x}_i, \mathbf{x}_j; \mathbf{z}_{ij})\|^2_{\mathbf{\Omega}_{ij}} \\ & + \sum_{(i,k)\in\mathcal{E}_{\mathrm{pa}}} \|\mathbf{r}^{\mathrm{a}}_{ik}(\mathbf{x}_i, \mathbf{x}_{\mathcal{C}_k}; \mathbf{z}_{ik})\|^2_{\mathbf{\Omega}_{ik}}.\end{aligned} \tag{3}$$

The first term corresponds to the standard LiDAR SLAM objective over robot poses, while the second term adds pose-camera constraints that connect robot poses to persistent infrastructure nodes. In the proposed method, each camera is represented by one node that is reused across all of its detections, and each visual detection of the robot contributes one pose-anchor edge. These additional terms couple distant parts of the trajectory through the same camera nodes and therefore counteract drift in sparse, repetitive, or featureless environments without requiring a separate post-hoc registration of camera locations onto an already drifted map.

C. System Overview

Figure 2 shows the overall workflow of the proposed camera-augmented pose-graph SLAM system. The robot runs a graph-based 2D LiDAR SLAM backend. A set of N ceiling- or wall-mounted cameras $\{\mathcal{C}_i\}_{i=1}^N$ observe the workspace from stable viewpoints. Unlike a post-processing pipeline in which cameras are first aligned to a finished SLAM map, the proposed method inserts each camera directly into the pose graph as a persistent node whose map-frame pose is refined jointly with the robot trajectory. A YOLO-based inference node [19] runs on each camera stream; when the robot is detected, the node calls the `add_camera_constraint` service to either initialise or update the corresponding camera node and inject a factor into the Ceres pose graph, thereby augmenting Eq. 3 with repeated camera-to-robot observations.

D. Constraint Formulation

For each camera-derived anchor edge $(t, i) \in \mathcal{E}_{\mathrm{cam}}$, the robot pose $\mathbf{x}_t$ is constrained relative to camera node $\mathbf{x}_{\mathcal{C}_i}$. The relative transform observed when the robot is detected is:

$$\hat{\mathbf{T}}_{\mathcal{C}_i \to \mathrm{robot}} = \mathbf{x}_{\mathcal{C}_i}^{-1} \cdot \mathbf{x}_t \tag{4}$$

The associated camera residual, which instantiates the generic anchor residual in Eq. 3, is:

$$\mathbf{r}^{\mathrm{cam}}_{ti}(\mathbf{x}_t, \mathbf{x}_{\mathcal{C}_i}) = \mathbf{\Sigma}_i^{-\frac{1}{2}} \left(\mathrm{Log}\left(\hat{\mathbf{T}}^{-1}_{\mathcal{C}_i \to \mathrm{robot}} \cdot \mathbf{x}_{\mathcal{C}_i}^{-1} \cdot \mathbf{x}_t \right) \right) \tag{5}$$

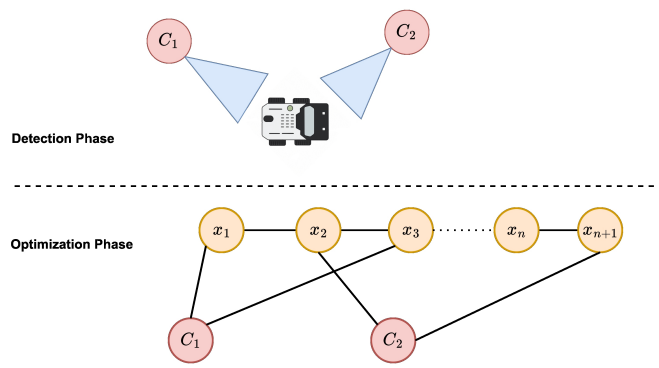


Fig. 2: Overview of the proposed framework. In the detection phase, fixed infrastructure cameras observe the robot from external viewpoints. In the optimisation phase, each observation adds a camera-to-robot relative pose constraint between a camera node $\mathcal{C}_i$ and a robot pose node $\mathbf{x}_t$, augmenting the LiDAR pose graph with repeated cross-time camera constraints.

where $\mathbf{\Sigma}_i = \mathrm{diag}(\sigma_x^2, \sigma_y^2, \sigma_\theta^2)$ encodes per-axis uncertainty and $\mathrm{Log}(\cdot)$ is the $SE(2)$ logarithmic map. This residual is inserted as a pose-camera term in the global objective, penalizing inconsistency between the current robot pose estimate, the current camera pose estimate, and the relative transform implied by the observation. Because the same camera node participates in multiple detections collected at different times, optimisation can refine that node jointly with the robot trajectory instead of treating the camera as a pre-fixed absolute reference.

E. Camera Node Initialisation and Refinement

Each camera node is created lazily when that camera produces its first valid detection of the robot. Given the current robot estimate $\mathbf{x}_t$ and the observed relative transform $\hat{\mathbf{T}}_{\mathcal{C}_i \to \mathrm{robot}}$, an initial map-frame estimate for the camera is obtained by inverting the measurement:

$$\mathbf{x}_{\mathcal{C}_i}^{(0)} = \mathbf{x}_t \cdot \hat{\mathbf{T}}_{\mathcal{C}_i \to \mathrm{robot}}^{-1} \tag{6}$$

This estimate is used only to initialise the parameter block and to associate future observations with a persistent camera identity. Every later detection from the same camera adds another residual of the form in Eq. 5, so the final camera pose is determined by the full set of robot-camera observations together with the LiDAR odometry and loop-closure constraints. In this way, the cameras become anchor-like nodes through repeated geometric consistency rather than through a manually entered pose on a completed map.

F. Covariance Selection

The information assigned to each camera observation determines how strongly a single detection can pull both the robot state and the associated camera node. We therefore attach uncertainty to the relative measurement itself rather than to a fixed map-frame camera prior. In all experiments, we set $\sigma_x = \sigma_y = 0.1\,\mathrm{m}$ and $\sigma_\theta = 0.05\,\mathrm{rad}$, which provides a conservative weighting for image-based robot localisation under pixel quantisation, projection error, and partial occlusion. This choice prevents one noisy detection from dominating the optimiser while still allowing repeated observations from the same camera to accumulate into a useful long-range constraint. Sensitivity of the results to these values is analysed in Section V.

G. Implementation

The inference node auto-discovers camera topics matching `*/image_raw` using periodic graph introspection. Constraints are throttled to one per camera per $5\,\mathrm{s}$ to avoid over-constraining the graph during bursts of detections. The `add_camera_constraint` service extends the standard Ceres-based SLAM problem in two ways. When a camera is observed for the first time, the service allocates a new camera parameter block initialised with Eq. 6. For every accepted detection, it adds a `CameraConstraintFunctor` cost function implementing Eq. 5 between the current robot pose node and that persistent camera node. In this way, the implementation realizes the graph augmentation described in Eq. 3 without replacing the existing LiDAR SLAM front-end or loop-closure pipeline, while allowing camera poses to be refined online together with the map.

IV. Experiments

A. Simulation Environment

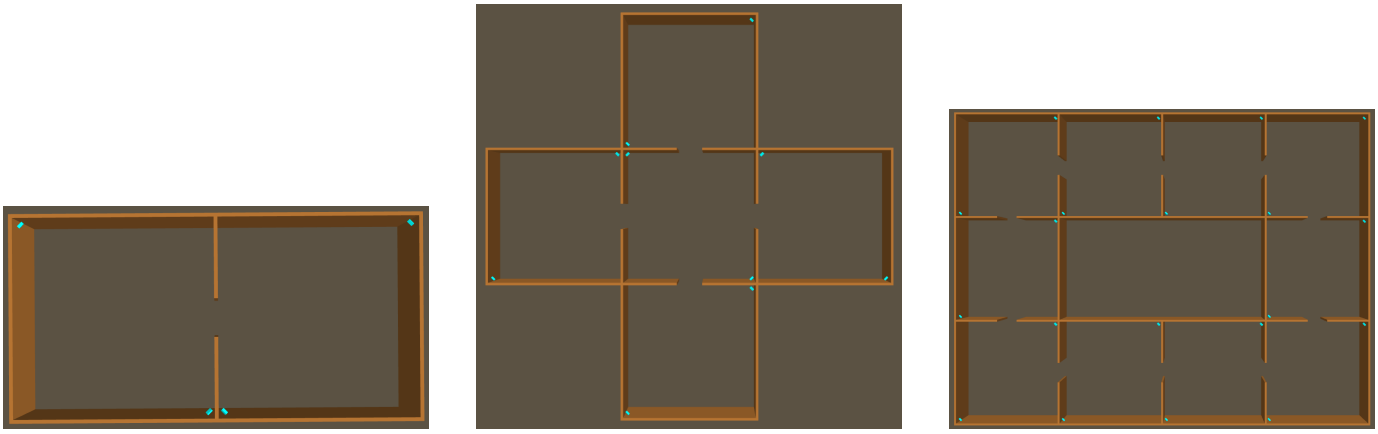
All experiments were conducted in Gazebo [20] running on Ubuntu 22.04. We evaluate the method in three indoor layouts shown in Figure 3: a 2-room horizontal layout, a 5-room cross layout, and a 10-room ring layout. In all scenarios, individual rooms measure $8\,\mathrm{m}\times 8\,\mathrm{m}$ and are connected by $4.75\,\mathrm{m}$-wide doorways. Two static cameras are mounted at the corners of each room at a height of $2.2\,\mathrm{m}$, with a 86° horizontal field of view.

B. Robot Platform

The simulated robot is an AgileX LIMO (Figure 4) with a 2D LiDAR (range $12\,\mathrm{m}$, 360°, $10\,\mathrm{Hz}$) and differential-drive kinematics. The robot is spawned at position $(x, y) = (4.0, 4.0)\ \mathrm{m}$ in the Gazebo world frame.

C. SLAM Configuration

We build on *SLAM Toolbox* [21], the de-facto standard 2D LiDAR SLAM framework in ROS 2. SLAM Toolbox has been widely adopted across diverse mobile-robot applications—from long-range autonomous navigation [6] to multi-floor building mapping and warehouse logistics—making it a representative and reproducible baseline for indoor SLAM research. It is run in synchronous mapping mode with the following parameters: map resolution $0.05\,\mathrm{m}/\mathrm{cell}$, scan-match every $0.5\,\mathrm{s}$, loop-closure minimum score 0.55. The Ceres solver [8] uses SPARSE_NORMAL_CHOLESKY with a Levenberg–Marquardt trust region.



(a) 2-room horizontal layout. (b) 5-room cross layout. (c) 10-room ring layout.

Fig. 3: Top-down Gazebo layouts used in the evaluation for the 2-room, 5-room, and 10-room scenarios. Blue diamonds indicate the fixed camera locations.

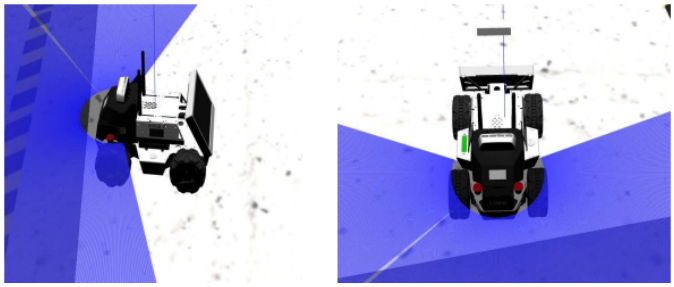


Fig. 4: The AgileX LIMO mobile robot used in simulation, shown from two viewpoints. The blue cone indicates the LiDAR field of view.

D. Baselines

1) **LiDAR-only SLAM**: the same SLAM backend with no camera constraints.
2) **Camera-constrained SLAM** (ours): identical configuration with camera constraints enabled ($\sigma_x = \sigma_y = 0.1$, $\sigma_\theta = 0.05$).

E. Metrics

We report:

- **ATE** (Absolute Trajectory Error) [22]: RMSE between estimated and ground-truth poses over the full trajectory.
- **Map consistency**: overlap integral between the produced OccupancyGrid and the ground-truth map (intersection over union of occupied cells).
- **Camera marker error**: Euclidean distance from the orange (estimated) camera marker to the yellow (GT) camera marker in RViz, averaged across all cameras at the end of one lap.

Each condition was run $N = 5$ times with randomised initial heading to assess variance.

TABLE I: Absolute Trajectory Error (ATE) across 5 runs (↓ better).

Method	Mean ATE (m)	Std (m)
Vanilla SLAM	–	–
Camera-constrained (ours)	–	–

(a) Vanilla SLAM. (b) Camera-constrained (ours).

Fig. 5: Occupancy maps after one full lap of the ring world. Ground-truth walls overlaid in red.

V. Results

A. Trajectory Accuracy

Table I summarises the Absolute Trajectory Error for baseline and constrained conditions. Camera constraints reduce mean ATE from **? m** $\pm$ **? m** to **? m** $\pm$ **? m**, a **?%** improvement.

B. Map Consistency

Figure 5 shows occupancy maps produced after one full lap of the ring environment. The baseline map shows ~**?%** overlap with ground truth, while the constrained map achieves **?%**.

C. Camera Marker Error

The mean Euclidean error between estimated (orange) and ground-truth (yellow) camera positions is reduced from **? m** to

Fig. 6: ATE vs. constraint standard deviation σ_x. Shaded region shows ± 1 standard deviation over 5 runs.

? m, confirming that the constraint injection correctly anchors the map frame.

D. Covariance Sensitivity

Figure 6 shows ATE as a function of σ_x for fixed $\sigma_\theta = 0.05$. Over-tightening ($\sigma_x < 0.05$ m) causes the solver to over-correct spurious detections, increasing error. Values in the range $[0.08, 0.15]$ m are consistently beneficial.

E. Discussion

The results demonstrate that even a small number of fixed cameras (one per room) provides sufficient anchoring to suppress drift in a loop-closing environment. The primary failure mode is false-positive detections when the robot is partially occluded; future work can address this with a confidence threshold on the YOLO detection score before injecting a constraint.

VI. Conclusion

We presented a method for reducing SLAM drift in indoor environments by injecting camera-to-robot pose constraints into the Ceres factor graph of a graph-based LiDAR SLAM backend. Fixed infrastructure cameras detect the robot visually via a YOLO model, and each detection provides an absolute pose anchor that supplements odometry and loop-closure edges. The approach is implemented as a standalone node requiring no modification to the SLAM solver core.

Experiments in a 10-room ring simulation show that enabling camera constraints reduces Absolute Trajectory Error by **?%** and improves map overlap with ground truth from **?%** to **?%**. These gains hold across randomised initial conditions and are robust to constraint covariance values in the range $\sigma_x \in [0.08, 0.15]$ m.

Several directions remain for future work. An important practical limitation of conventional infrastructure-assisted SLAM is that camera poses are often registered only after a map has already been created, which means any residual map drift is baked into the camera anchors themselves. This motivates formulations in which map structure and camera poses are refined together rather than in separate stages. First, replacing the world-file ground-truth camera pose with a visual relative-pose estimator (homography or PnP) will remove the dependency on externally specified camera positions. Second, extending the formulation to multiple robots with shared infrastructure cameras could benefit collaborative mapping. Third, an online covariance estimator based on detection confidence would make the method self-tuning.

References

[1] M. Montemerlo and S. Thrun, "Simultaneous localization and mapping with unknown data association using FastSLAM," in *Proceedings of the 2003 IEEE International Conference on Robotics and Automation (ICRA)*, 2003, pp. 1985–1991.

[2] N. Sunderhauf, S. Lange, and P. Protzel, "Using the unscented kalman filter in mono-SLAM with inverse depth parametrization for autonomous airship control," in *2007 IEEE International Workshop on Safety, Security and Rescue Robotics*, 2007, pp. 1–6.

[3] J. Zhu, H. Li, and T. Zhang, "Camera, LiDAR, and IMU based multi-sensor fusion SLAM: A survey," *Tsinghua Science and Technology*, vol. 29, no. 2, pp. 415–429, 2024.

[4] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based SLAM," *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.

[5] N. S"underhauf, "Robust optimization for simultaneous localization and mapping," Ph.D. dissertation, Chemnitz University of Technology, 2012. [Online]. Available: http://nbn-resolving.de/urn:nbn:de:bsz:ch1-qucosa-86443

[6] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725.

[7] F. Yang, S. Yamao, I. Kusajima, A. Moteki, S. Masui, and S. Jiang, "YOWO: You only walk once to jointly map an indoor scene and register ceiling-mounted cameras," 2025. [Online]. Available: https://arxiv.org/abs/2511.16521

[8] S. Agarwal, K. Mierle, and The Ceres Solver Team, "Ceres Solver," 2023. [Online]. Available: http://ceres-solver.org

[9] H. Taira, M. Okutomi, T. Sattler, M. Cimpoi, M. Pollefeys, J. Sivic, T. Pajdla, and A. Torii, "InLoc: Indoor visual localization with dense matching and view synthesis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7199–7209.

[10] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk, "From coarse to fine: Robust hierarchical localization at large scale," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12 716–12 725.

[11] G. Lee, T. Shibata, K. Nonaka, and S. Nobuhara, "Extrinsic camera calibration from a moving person," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 629–10 636, 2022.

[12] M. P"atzold, S. Bultmann, and S. Behnke, "Online marker-free extrinsic camera calibration using person keypoint detections," in *Pattern Recognition*, 2022, pp. 311–325.

[13] R. Khirodkar, A. Bansal, L. Ma, R. Newcombe, and O. Hilliges, "Ego-Humans: An egocentric 3d multi-human benchmark," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 19 845–19 857.

[14] Y. Liu, C. He, C. Yuen, and B. W. M. Lau, "Cost-effective mapping of mobile robot based on the fusion of UWB and short-range 2-d LiDAR," *IEEE/ASME Transactions on Mechatronics*, vol. 27, no. 4, pp. 2557–2568, 2022.

[15] S. Liu, J. Lin, Y. Wang, and X. Kong, "A method for non-line of sight identification and delay correction for UWB indoor positioning," in *2022 IEEE 17th Conference on Industrial Electronics and Applications (ICIEA)*, 2022, pp. 1595–1600.

[16] Y. Jiang, Y. Cheng, S. Dai, and Y. Wan, "Gps-denied LiDAR-based SLAM—a survey," *IET Cyber-Systems and Robotics*, 2025.

[17] Z. Jiang, H. Yin, X. Jin, and Y. Tian, "A simultaneous localization and mapping (SLAM) framework for 2.5d map building based on low-cost LiDAR and vision fusion," *Applied Sciences*, vol. 9, no. 10, p. 2105, 2019.

[18] Y. Chen, L. Zhao, Y. Zhang, S. Huang, and G. Dissanayake, "Anchor selection for SLAM based on graph topology and submodular optimization," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 102–118, 2022.

[19] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[20] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *IEEE/RSJ IROS*, 2004, pp. 2149–2154.

[21] S. Macenski and I. Jambrecic, "SLAM toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[22] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *IEEE/RSJ IROS*, 2012, pp. 573–580.